



OneHome System Anatomical Outline

⚙ Status	Not started
☑ Complete?	☐

Home Service App — System Design from First Principles

1. Fundamental Truth of the System

A home service app connects a customer who needs a service at a particular location with a service provider who can perform that service under agreed conditions.

At its core, the system must solve five fundamental problems:

1. Demand: What service does the customer need, and where?
2. Supply: Which service providers have the required skills, availability, and geographic coverage?
3. Agreement: Can both parties agree on the scope, price, schedule, and terms?
4. Execution: Can the service be delivered and verified?
5. Settlement: Can payment, provider compensation, and any disputes be resolved correctly?

Everything else—user profiles, search filters, notifications, ratings, dashboards, and analytics—exists to support these five problems.

The system's fundamental transaction

A home service transaction can be represented as:

Request → Match → Agreement → Service → Settlement

This is the central lifecycle around which the system should be designed.

Working assumption: The application is a two-sided marketplace for services such as plumbing, electrical repairs, cleaning, appliance repair, and other household maintenance. Customers book independent service providers through a centralized platform.

The system design below focuses on the core marketplace rather than assuming that the platform employs the service providers directly.

2. System Requirements

System requirements define what the application must accomplish and the constraints under which it must operate.

2.1 Functional requirements

These are capabilities the system must provide.

ID	Requirement	Purpose
FR-01	User registration and authentication	Establish customer and provider identities.
FR-02	Service catalog management	Define available services, categories, and descriptions.
FR-03	Provider profile management	Store skills, service areas, pricing, and credentials.
FR-04	Service request creation	Capture service type, location, schedule, and problem description.
FR-05	Provider discovery and matching	Identify eligible providers based on service, location, and availability.
FR-06	Booking and scheduling	Reserve a provider and establish an agreed appointment.
FR-07	Quote and pricing management	Communicate estimated or agreed service costs.
FR-08	Booking lifecycle management	Track confirmation, progress, completion, cancellation, and disputes.

FR-09	Payment processing	Collect customer payments and track payment status.
FR-10	Provider payouts	Record and initiate provider compensation.
FR-11	Notifications and messaging	Communicate booking updates and coordinate service delivery.
FR-12	Reviews and ratings	Allow customers to evaluate completed services.
FR-13	Dispute and refund management	Handle disagreements, refunds, and service issues.
FR-14	Administrative management	Manage users, providers, service categories, and operational issues.

2.2 Non-functional requirements

These requirements describe the system's quality attributes and operational constraints.

Requirement	Engineering objective
Reliability	Prevent lost bookings, payments, and critical state changes.
Consistency	Maintain a single valid booking state and accurate financial records.
Security	Protect identities, addresses, payment references, and account access.
Availability	Keep essential booking and service-management functions accessible.
Performance	Return search results and booking responses within acceptable latency targets.
Scalability	Support growth in users, providers, bookings, and geographic coverage.
Observability	Record logs, metrics, and traces to diagnose failures.
Maintainability	Keep business rules understandable and testable.
Privacy	Restrict access to customer and provider information based on legitimate need.
Recoverability	Recover from service outages, interrupted workflows, and data failures.

First-principles distinction

A requirement is not a technology choice.

For example, "prevent double booking" is a requirement. Using a distributed lock, a database constraint, or a transactional reservation mechanism is an implementation decision.

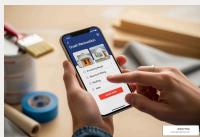
Similarly, "secure payments" is a requirement. Using a particular payment gateway is a design choice.

The architecture should follow the required behavior, not the other way around.

3. Actors

An actor is an entity outside the system boundary that interacts with the application to achieve a goal.

The key distinction is between who requests the service, who fulfills it, and who governs the transaction.



1. Customer

The person requesting a home service.

Responsibilities:

- Browse services and providers.
- Submit service requests.
- Compare quotes and select providers.
- Schedule, pay for, and review services.
- Cancel bookings or report problems.



2. Service Provider

The individual or business performing the requested service.

Responsibilities:

- Maintain a profile, service offerings, and availability.
- Receive and respond to service requests.
- Submit quotes and accept bookings.
- Perform the service and report completion.

- Receive compensation.



3. Administrator / Operations Staff

The actor responsible for platform governance and operational intervention.

Responsibilities:

- Verify providers and manage accounts.
- Maintain service categories and platform rules.
- Investigate disputes and manage refunds.
- Monitor suspicious activity and operational issues.

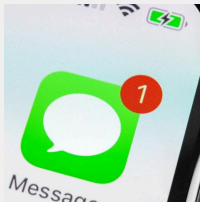


4. Payment Gateway

An external system that processes payment operations.

Responsibilities:

- Authorize or capture payments.
- Report payment success or failure.
- Support refunds and payment reconciliation.



5. Notification Service

An external delivery system for push notifications, SMS, or email.

Responsibilities:

- Deliver booking confirmations and reminders.
- Notify customers and providers about important changes.

Actor boundary: The customer, provider, and administrator are human actors. The payment gateway and notification service are external system actors. The database, matching engine, and booking manager are internal components, not actors.

4. Top Use Cases

A use case describes a goal an actor accomplishes through the system.

The most important use cases should map directly to the fundamental transaction lifecycle.

4.1 Core use cases

ID	Use case	Primary actor	Expected outcome
UC-01	Register / Log In	Customer, Provider	Authenticated account
UC-02	Browse Services	Customer	Selected service category
UC-03	Submit Service Request	Customer	Valid service request
UC-04	Find Matching Providers	Customer / System	Eligible provider list
UC-05	Submit Quote	Provider	Proposed price and terms
UC-06	Accept Quote and Book	Customer	Confirmed booking
UC-07	Accept or Decline Request	Provider	Provider response recorded
UC-08	Reschedule or Cancel Booking	Customer, Provider	Updated booking status
UC-09	Perform Service	Provider	Service delivered
UC-10	Confirm Completion	Customer, Provider	Completion recorded
UC-11	Pay for Service	Customer	Payment successfully recorded
UC-12	Process Provider Payout	System / Payment Gateway	Provider compensation initiated
UC-13	Submit Review	Customer	Review recorded
UC-14	Resolve Dispute	Administrator	Dispute decision and resolution recorded

4.2 Primary use case: Book a home service

UC-06 — Accept Quote and Book

Primary transaction

Primary actor: Customer

Supporting actor: Service Provider

Preconditions

- The customer is authenticated.
- A valid service request exists.
- At least one provider has submitted a valid quote.
- The quote has not expired or been withdrawn.

Main flow

1. The customer reviews a provider's quote.
2. The customer selects an available appointment.
3. The system validates the quote, provider availability, and booking conditions.
4. The system reserves the appointment.
5. The system records the agreed price and terms.
6. The booking is confirmed.
7. Both parties receive a confirmation.

Alternative flows

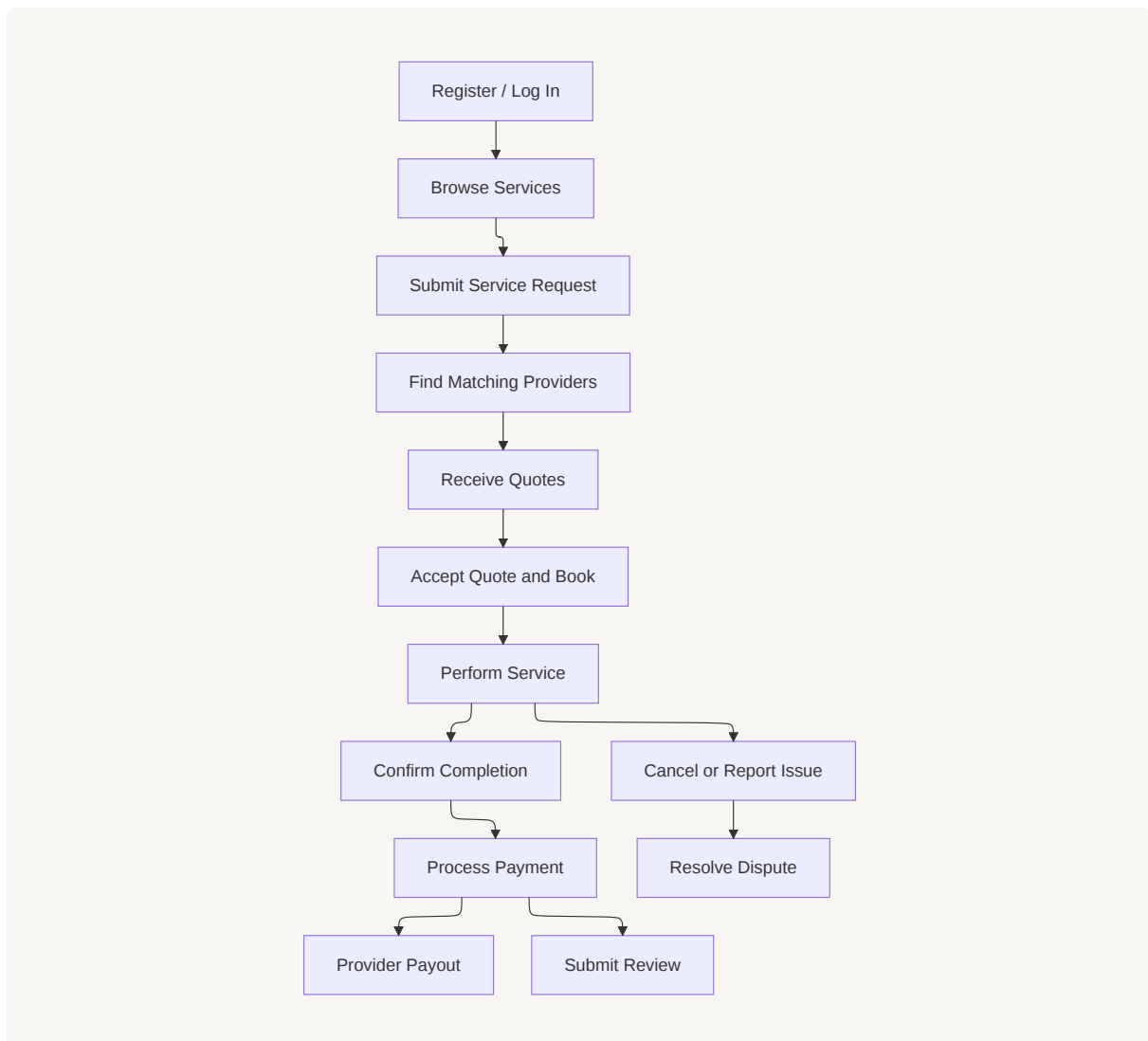
- If the provider is no longer available, the customer selects another slot.
- If the quote has expired, the customer requests a new quote.
- If payment authorization is required and fails, the booking remains unconfirmed or is released.
- If the customer cancels before confirmation, the request remains unbooked.

Postconditions

A confirmed booking exists with an agreed provider, service, appointment, and price.

4.3 Use case dependency structure

The dependencies below show the logical order of the core workflow.



This is a simplified dependency chain, not a strict requirement that every booking must have a quote, a customer completion confirmation, or a review. Fixed-price services and alternative completion workflows can follow different paths.

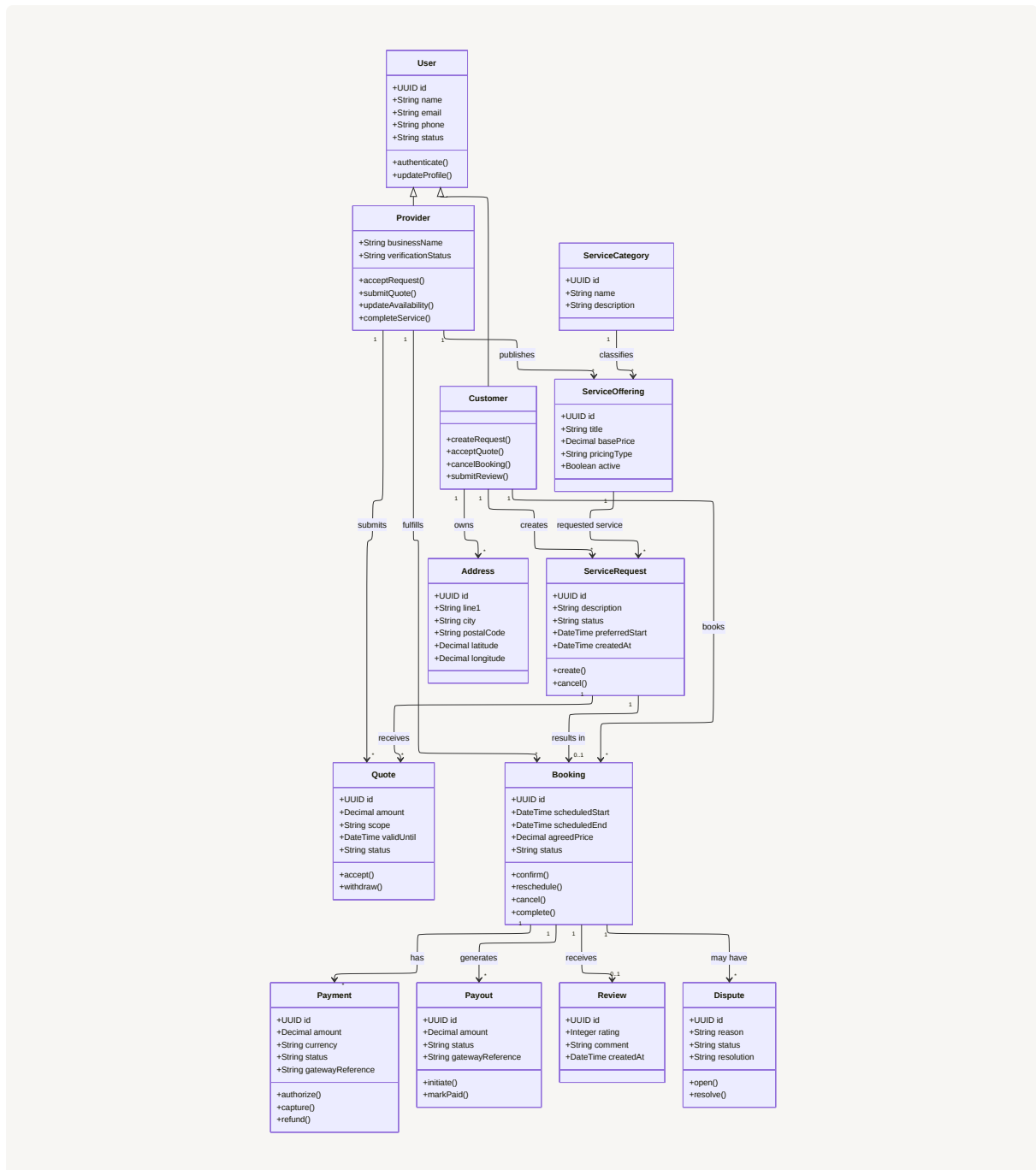
5. Class Diagram

The class diagram defines the core domain entities, their attributes, their responsibilities, and their relationships.

The central modeling principle is that a booking is a durable agreement between a customer and a provider, while a payment is a separate financial transaction associated with that agreement.

This separation prevents changes to booking status from silently altering financial history.

5.1 Core domain model



5.2 Relationships explained

User → Customer / Provider: Both roles share identity and authentication information. A user may hold both roles if the product permits it.

ServiceOffering → ServiceRequest: A provider's offering defines the service that can be requested. The request should retain a snapshot of relevant service details so later catalog changes do not rewrite historical transactions.

ServiceRequest → Quote: Multiple providers may submit competing quotes for the same request.

ServiceRequest → Booking: A request may produce at most one confirmed booking in this simplified model. If the product allows multiple providers per request, this relationship must be redesigned.

Booking → Payment: One booking can have multiple payment records, for example, an initial authorization, a final capture, and a refund.

Booking → Payout: Provider compensation is recorded separately from customer payment, allowing platform fees, adjustments, and payout failures to be handled explicitly.

Booking → Review / Dispute: A review or dispute is tied to a particular service transaction rather than to a generic user interaction.

5.3 Important design constraints

The class diagram describes conceptual relationships. The implementation must enforce the following invariants:

- A confirmed booking must reference a valid customer, provider, service, appointment, and agreed price.
- A provider must not have overlapping confirmed bookings when the service requires exclusive time allocation.
- A quote can be accepted only once.
- A payment must not be captured beyond the authorized or otherwise permitted amount.
- A payout must not exceed the amount the provider is entitled to receive.
- A review must be tied to an eligible completed booking.
- Every financial adjustment must preserve an auditable record.

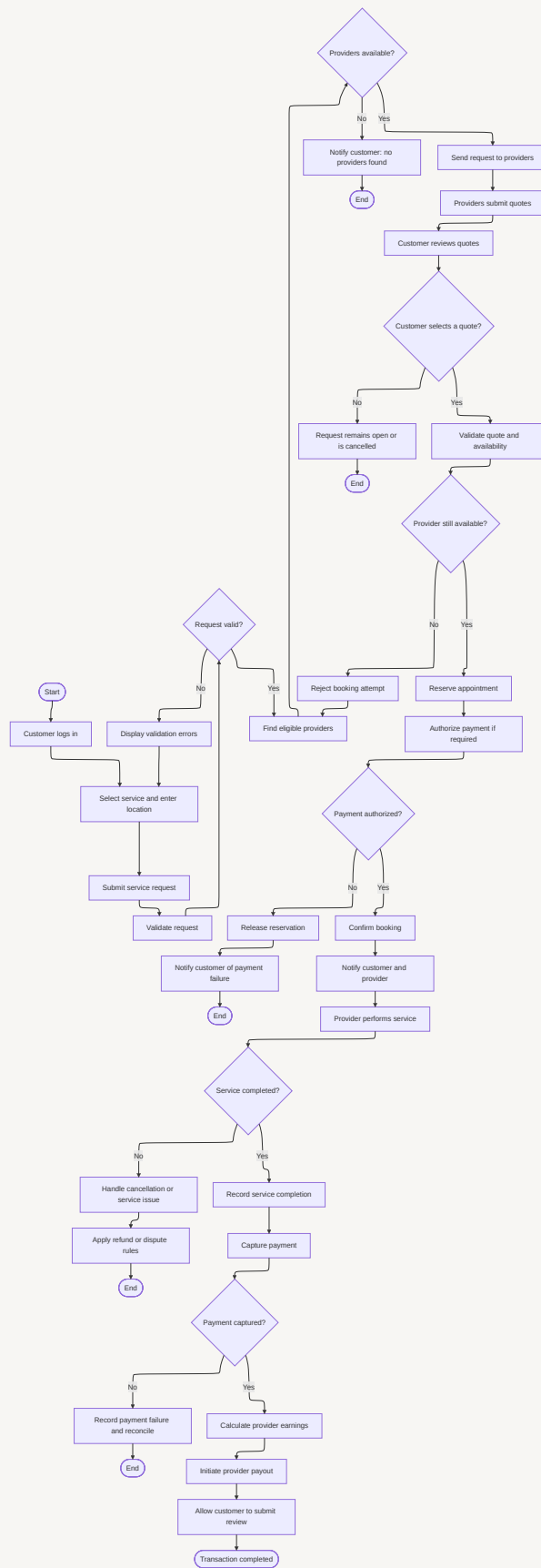
These constraints are more fundamental than the choice of database, framework, or deployment model.

6. Activity Diagram

The activity diagram describes the operational workflow from the moment a customer requests a service until the booking is completed and payment is settled.

It must account for both the successful path and failure conditions, because real transactions can fail at multiple points.

6.1 Main activity diagram: End-to-end service booking

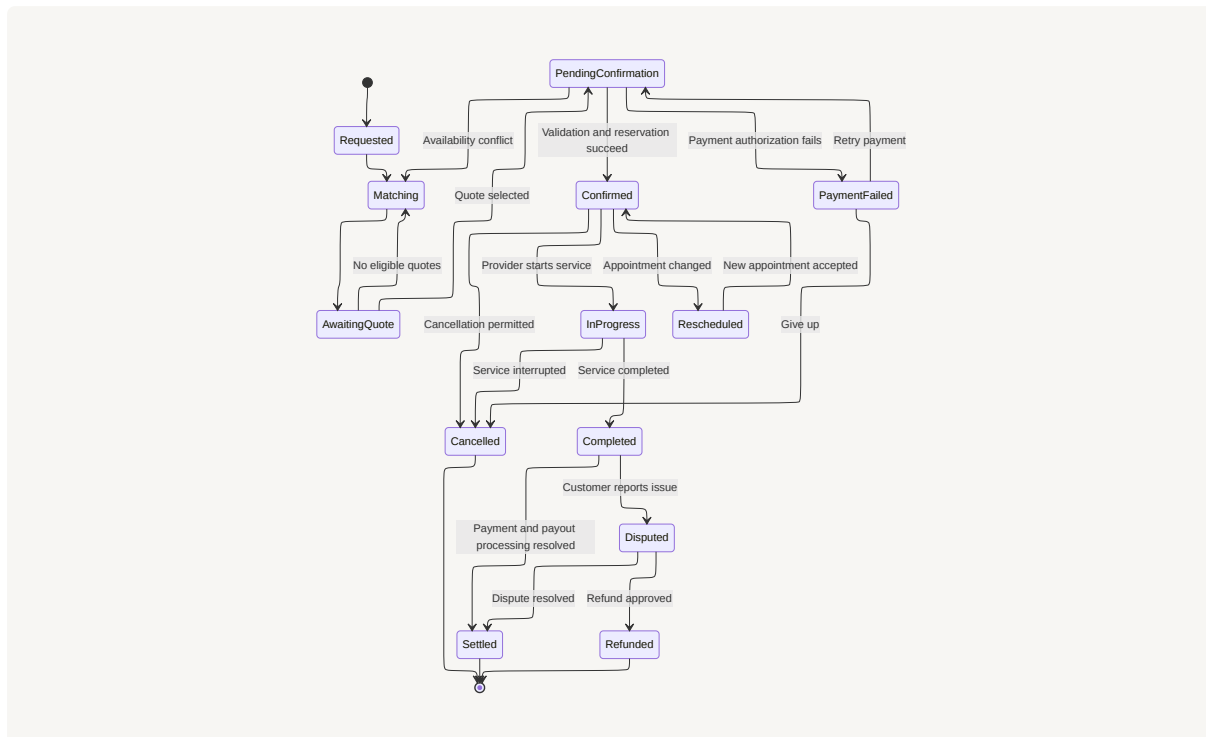


6.2 Critical decision points

Decision	Why it matters
Request valid?	Request valid?
Providers available?	Prevents the system from offering a service that cannot be fulfilled.
Provider still available?	Prevents competing customers from booking the same provider's time slot.
Payment authorized?	Determines whether the booking can be confirmed under the chosen payment policy.
Service completed?	Determines whether the transaction proceeds to settlement or exception handling.
Payment captured?	Determines whether the platform has successfully collected the agreed amount.

6.3 Booking state machine

An activity diagram explains the workflow; a state machine defines which state transitions are permitted.

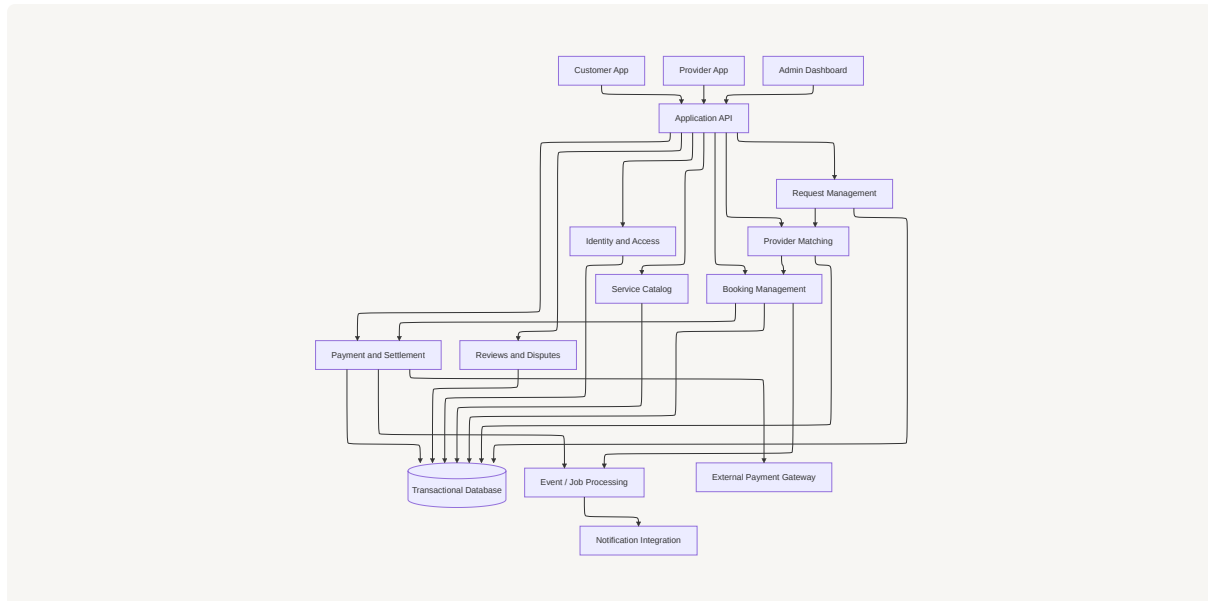


Important: The booking state and payment state should be modeled separately. For example, a booking may be completed while its payment is still being

reconciled. A refund should also be recorded as a financial operation rather than simply changing the booking to a generic cancelled state.

7. Logical System Components

The domain model and workflows imply a set of internal components. These are responsibilities, not necessarily separate microservices.



A practical first implementation can use a modular monolith: one deployable application with clear internal modules and a transactional database. This is sufficient to preserve domain boundaries without introducing distributed-system complexity prematurely.

The components can be separated into independent services later if actual workload, reliability, or organizational constraints justify it.

8. First-Principles Design Summary

The entire system can be reduced to three core responsibilities:

1. Coordinate people and resources: Match customer demand with provider capabilities and availability.
2. Enforce agreements: Preserve the service scope, schedule, price, and permitted state transitions.
3. Preserve transaction integrity: Record service delivery, payment, payout, and dispute outcomes without losing financial or operational history.

The central engineering challenge is not simply allowing a customer to book a service. It is ensuring that the customer and provider share the same valid agreement, that the provider's time is not double-booked, and that the financial records remain correct even when the workflow is interrupted.

That is the foundation of a reliable home service marketplace.